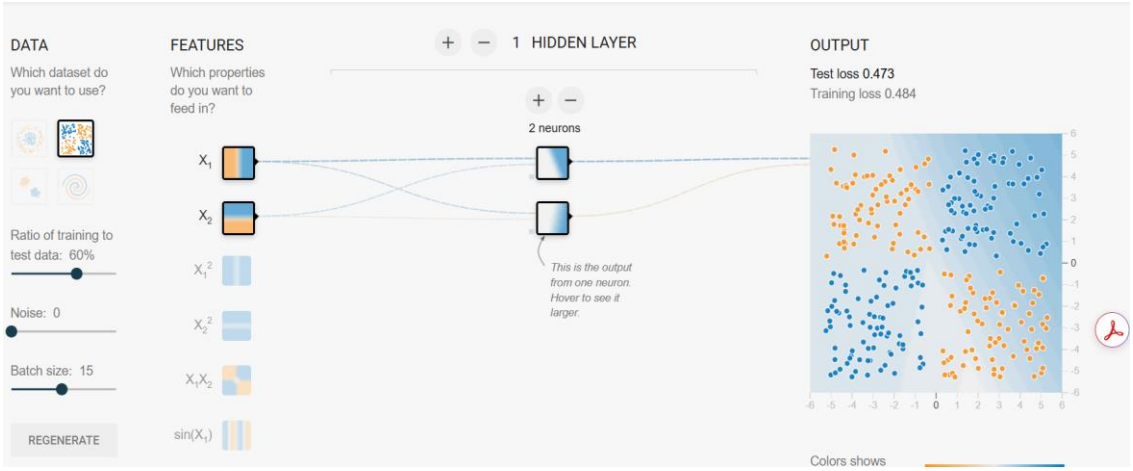
Question # 1: Try various settings for the number of layers and neurons using ReLU as activation function.

Architecture comparison:

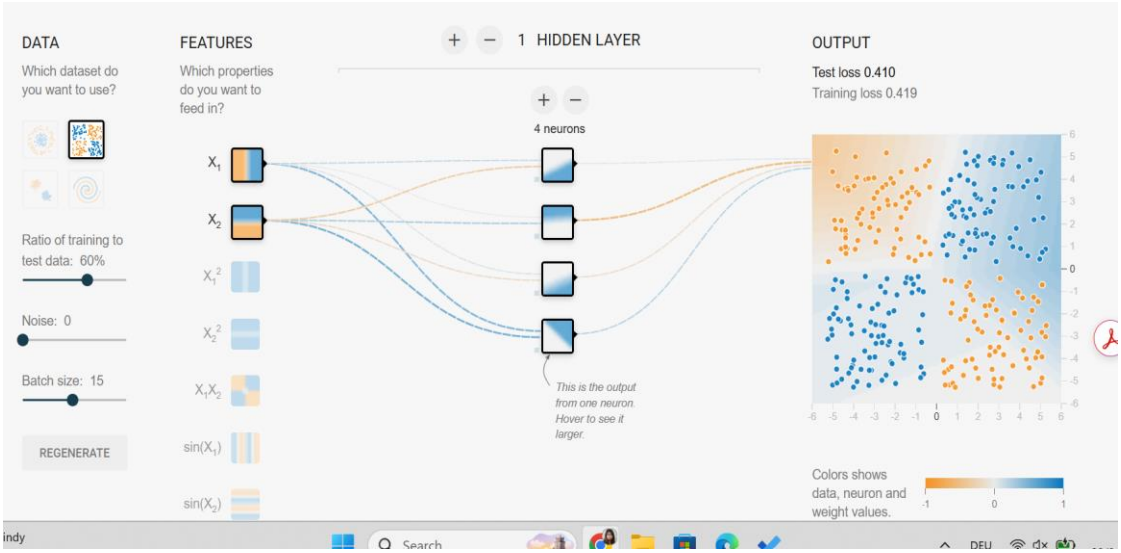
The checkerboard pattern is non-linearly separable. It requires learning $x_1 \cdot x_2$, a product interaction, which a linear model cannot express. ReLU neurons must be stacked to approximate this.

1. **1 Hidden layer, 2 neurons:**



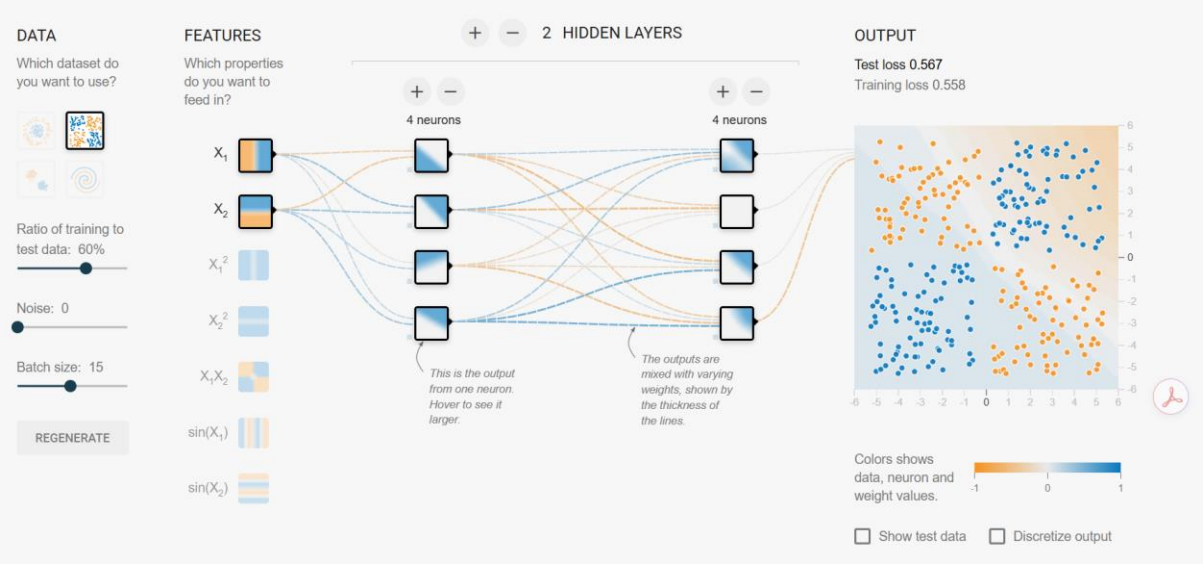
Result: poor fit. Insufficient capacity to represent all four quadrants.

2. **1 hidden layer, 4 neurons**



Result: marginal. Occasionally converges but sensitive to random initialization.

3. **2 hidden layers, 4 neurons each**



Result: Good — reliable convergence.

The first layer learns linear half-planes from x1 and x2; the second layer combines these into quadrant-selective regions. Four neurons per layer provides enough linear pieces to tile all four checkerboard quadrants. This architecture reliably converges.

Question 2: Variability across repeated training runs

Running the same network architecture with the same hyperparameters multiple times produces noticeably different results. Three types of variation were observed:

Observation	Detail
Different loss curves	Some runs converge within ~100 steps; others plateau for hundreds of steps before improving or getting stuck entirely.
Different decision boundaries	The learned pattern can appear rotated or mirrored. Many distinct weight configurations can produce equivalent predictions on this symmetric dataset (permutation symmetry of neurons).
Occasional complete failures	Especially with small networks (1 layer), gradient descent can become trapped in local minima where the network predicts one class everywhere.

Neural networks minimize a complex loss with many hills and flat regions.
Because weights start randomly, training can end in different solutions or get stuck.

This is normal and handled by:

- running training multiple times,
- using good initialization (Xavier/He),
- or using optimizers with momentum.

Question 3: Most helpful additional input feature

Adding engineered features to the input can dramatically reduce the work the network must do internally. The following features were evaluated on the checkerboard dataset:

Feature	Usefulness	Reason
x 1 · x2	Best	Directly encodes quadrant sign interaction. The exact structure of the checkerboard. Enables convergence with minimal network depth.
(x 1)^2 . (x2)^2	Moderate	Provides radial distance but not sign interaction. Reduces symmetry, mildly helpful.
sin(x1), sin(x2)	Moderate	Introduces periodicity. Useful when scale matches data, but effect is inconsistent.

The feature $x1 \cdot x2$ is very important because it directly tells the class.
If the product is positive → class A, if negative → class B.

Since this feature is given, even a simple model (one neuron) can solve the problem.
This shows that good feature design can make learning easier and more stable